

Chapter3 Notes

Unit III: HTML5

[7 Hrs.]

1. HTML5 APIs

HTML5 introduced JavaScript APIs that work directly with browser — no plugins (Flash) needed.

Canvas API

- `<canvas>` — draws graphics (shapes, charts, games) via JavaScript.
- Real project use: chart libraries (Chart.js), signature pads, drawing apps, game engines.

```
<canvas id="myCanvas" width="300" height="150"></canvas>
<script>
  const ctx = document.getElementById("myCanvas").getContext("2d");
  ctx.fillStyle = "#2563eb";
  ctx.fillRect(20, 20, 150, 80); // draws a blue rectangle
</script>
```

Other common HTML5 APIs (overview)

API	Purpose	Real use
Geolocation API	Get user's location	Food delivery apps showing nearby restaurants
Web Storage (<code>localStorage</code> / <code>sessionStorage</code>)	Store data in browser	Cart items, dark-mode preference, auth token
Drag and Drop API	Drag elements	File upload boxes (drag file to upload)
Canvas API	Draw graphics	Charts, games, image editors

Geolocation API

- Real project use: food delivery apps (find nearby restaurants), store-locator pages, weather apps.

```
<button onclick="getLocation()">Find My Location</button>
<p id="locationOutput"></p>

<script>
function getLocation() {
  if (navigator.geolocation) {
    navigator.geolocation.getCurrentPosition(showPosition, showError);
  } else {
    document.getElementById("locationOutput").innerHTML = "Geolocation not supported.";
  }
}

function showPosition(position) {
  const lat = position.coords.latitude;
  const lon = position.coords.longitude;
  document.getElementById("locationOutput").innerHTML =
    `Latitude: ${lat}, Longitude: ${lon}`;
}

function showError() {
  document.getElementById("locationOutput").innerHTML = "Unable to retrieve location.";
}
</script>
```

Web Storage API (localStorage / sessionStorage)

- `localStorage` — persists even after browser closed (cart items, theme preference).
- `sessionStorage` — cleared when tab/browser closes (temporary form data, one-session login state).

```
<!-- Real project: dark mode toggle saved across visits -->
<button onclick="toggleDarkMode()">Toggle Dark Mode</button>

<script>
function toggleDarkMode() {
  document.body.classList.toggle("dark");
  const isDark = document.body.classList.contains("dark");
  localStorage.setItem("darkMode", isDark); // persists after refresh/close
}

// On page load, apply saved preference
if (localStorage.getItem("darkMode") === "true") {
```

```
document.body.classList.add("dark");
}
</script>
```

```
<!-- sessionStorage example: multi-step checkout form, cleared when tab closes -->
<script>
  sessionStorage.setItem("shippingAddress", "Kathmandu, Nepal");
  const address = sessionStorage.getItem("shippingAddress");
</script>
```

Drag and Drop API

- Real project use: file upload boxes (“drag file here to upload”), Trello-style task boards, image reordering.

```
<div id="dropZone" style="width:300px;height:150px;border:2px dashed #999;text-align:center;line-height:150px;">

  Drop file here
</div>

<script>
const dropZone = document.getElementById("dropZone");

dropZone.addEventListener("dragover", (e) => {
  e.preventDefault(); // required to allow drop
  dropZone.style.borderColor = "blue";
});

dropZone.addEventListener("drop", (e) => {
  e.preventDefault();
  const file = e.dataTransfer.files[0];
  dropZone.innerHTML = `File dropped: ${file.name}`;
});
</script>
```

2. HTML5 Forms

HTML5 added new **input types** and **validation attributes** — reduces need for JavaScript validation.

New Input Types

```

<!-- Real project: booking/checkout form -->
<label>Email:</label>
<input type="email" name="email" required>

<label>Website:</label>
<input type="url" name="website">

<label>Appointment Date:</label>
<input type="date" name="appt_date">

<label>Appointment Time:</label>
<input type="time" name="appt_time">

<label>Quantity (1-10):</label>
<input type="number" min="1" max="10" name="qty">

<label>Volume:</label>
<input type="range" min="0" max="100" name="volume">

<label>Search:</label>
<input type="search" name="q">

<label>Phone:</label>
<input type="tel" name="phone" pattern="[0-9]{10}">

<label>Pick Color:</label>
<input type="color" name="theme_color">

```

Type	Use case
email	Validates @ format automatically
url	Validates URL format
date / time	Shows native date/time picker (booking sites, event forms)
number	Numeric input with min / max / step (quantity in cart)
range	Slider (volume control, price range filter on e-commerce site)
tel	Phone number, works with pattern
color	Native color picker (theme customizer)
search	Search box styling + clear button

Built-in Validation Attributes

Attribute	Purpose	Example
<code>required</code>	Field must not be empty	<code><input required></code>
<code>placeholder</code>	Hint text	<code>placeholder="Enter email"</code>
<code>min</code> / <code>max</code>	Numeric/date bounds	<code>min="1" max="10"</code>
<code>minlength</code> / <code>maxlength</code>	Text length bounds	<code>maxlength="20"</code>
<code>pattern</code>	Regex validation	<code>pattern="[A-Za-z]{3,}"</code>

Real project example — signup form with validation, no JS needed:

```
<form action="/signup" method="POST">
  <input type="text" name="username" required minlength="4" maxlength="15"
    placeholder="Username">
  <input type="email" name="email" required placeholder="Email">
  <input type="password" name="password" required minlength="8"
    placeholder="Password">
  <input type="tel" name="phone" pattern="[0-9]{10}" placeholder="10-digit phone">
  <button type="submit">Sign Up</button>
</form>
```

Browser blocks submission and shows native error message if any rule fails — this is exactly how most sign-up forms validate before even hitting the server.

3. Responsive Web Design

- Website automatically adapts layout to different screen sizes (mobile, tablet, desktop).
- Core technique used in **every modern real-world website** (Amazon, Facebook, any company site).

Key ingredient: Viewport meta tag

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

Without this, mobile browsers render page at desktop width then shrink it — text becomes unreadably small.

Techniques

- Fluid/relative units: `%`, `em`, `rem`, `vw`, `vh` instead of fixed `px`.
- Flexible images: `img { max-width: 100%; height: auto; }`

- **Media queries** (detailed under CSS chapter) to change layout per screen width.

Real project example — responsive navbar (stacks on mobile):

```
.navbar { display: flex; justify-content: space-between; }

@media (max-width: 600px) {
  .navbar { flex-direction: column; }
}
```

4. HTML5 (Overview / What's New)

HTML5 (released 2014) added features beyond old HTML4: - New **semantic tags**: `<header>`, `<nav>`, `<main>`, `<section>`, `<article>`, `<aside>`, `<footer>`. - New **form input types** (see section 2). - **Multimedia** support natively: `<audio>`, `<video>` (no Flash plugin needed). - **Canvas & SVG** for graphics. - **APIs**: Geolocation, Web Storage, Drag & Drop, Web Workers. - Cleaner, simpler doctype: `<!DOCTYPE html>` (vs long HTML4 doctype string).

Why it matters in real projects: modern browsers, frameworks (React, Vue, Angular) all render into HTML5 markup — semantic tags + native form validation + multimedia are foundation of any current website.

5. Semantic Markup

(Reinforced from Chapter 2 — critical enough to repeat/expand in HTML5 context.)

- Semantic tags describe **meaning** of content, not just appearance.
- Improves **SEO** (Google understands page structure), **accessibility** (screen readers), **maintainability**.

Real project example — full page layout using only semantic tags:

```
<body>
  <header>
    <h1>ShopEasy</h1>
    <nav>
      <a href="/">Home</a>
      <a href="/shop">Shop</a>
      <a href="/cart">Cart</a>
    </nav>
  </header>
```

```

<main>
  <section id="featured-products">
    <h2>Featured Products</h2>
    <article>
      <h3>Wireless Mouse</h3>
      <p>Rs. 1,500</p>
    </article>
  </section>

  <aside>
    <h3>Filters</h3>
    <p>Price range, brand, rating...</p>
  </aside>
</main>

<footer>
  <p>&copy; 2026 ShopEasy. All rights reserved.</p>
</footer>
</body>

```

Non-semantic vs semantic:

```

<!-- Old/bad way -->
<div class="header">...</div>
<div class="nav">...</div>

<!-- HTML5 semantic way -->
<header>...</header>
<nav>...</nav>

```

6. Best Practices and Optimization

Practices followed in professional/production codebases:

1. **Proper indentation & formatting** — nested tags indented consistently (2 or 4 spaces), improves readability for team collaboration.
2. **Consistent naming conventions** — `class="product-card"` (kebab-case), not `class="ProductCard1"`.
3. **Use semantic tags** instead of `<div>` / `<span>` everywhere.
4. **Always include `alt`** on images (accessibility + SEO).
5. **Minimize inline styles** — keep CSS in external files for maintainability.
6. **Optimize images** (compressed formats like WebP) — faster page load.
7. **Validate HTML** (W3C validator) — catch unclosed tags, errors.
8. **Use `<meta charset="UTF-8">` and viewport tag** in every page.

9. **Lazy-load images** (`loading="lazy"`) for performance on long pages.
10. **Accessibility**: use `<label for="">` , ARIA attributes where needed for screen readers.

Example — optimized image tag used in real projects:

```

```

Quick Recap (Exam Points)

- HTML5 APIs: Canvas (drawing), Geolocation, Web Storage, Drag & Drop.
- New input types: `email` , `url` , `date` , `time` , `number` , `range` , `tel` , `color` .
- Validation attributes: `required` , `pattern` , `min / max` , `minlength / maxlength` — built-in, no JS needed.
- Responsive design = layout adapts to screen size; viewport meta tag + media queries + fluid units.
- HTML5 = semantic tags + native multimedia (`audio / video`) + Canvas/SVG + new APIs + simpler doctype.
- Semantic markup improves SEO, accessibility, maintainability.
- Best practices: indentation, naming conventions, `alt` text, external CSS, image optimization, validation.

Possible Exam Questions

Short Answer / Conceptual

1. What is Canvas API? Write a short example to draw a rectangle.
2. List and explain any five new input types introduced in HTML5.
3. What is the use of `pattern` and `required` attributes in HTML5 forms?
4. What is responsive web design? Why is the viewport meta tag necessary?
5. What is the difference between HTML4 and HTML5? (mention any 4 differences)
6. What are HTML5 APIs? Explain Geolocation API and Web Storage API briefly.
7. Explain the importance of semantic markup with examples.
8. List any five best practices to follow while writing HTML code.
9. What is the difference between `localStorage` and `sessionStorage` ?
10. What is the purpose of `loading="lazy"` attribute on images?

Long Answer / Programming

1. Design a booking form using HTML5 input types: `date`, `time`, `email`, `tel`, `number` with proper validation attributes.
2. Write HTML5 code using Canvas API to draw a simple shape (rectangle/circle) with color fill.
3. Design a fully semantic webpage layout (header, nav, main with section/article, aside, footer) for an online shop's homepage.
4. Explain and demonstrate with code how media queries help in building responsive websites.
5. Write an HTML5 form for user registration using at least 6 different HTML5 input types and validation attributes.
6. Compare old (`<div class="header">`) vs new (`<header>`) approach with a full example page using both, and explain which is better and why.

True/False or Fill in the Blanks

1. `<canvas>` tag requires JavaScript to actually draw anything. (True)
2. `type="date"` input shows a native date-picker in supporting browsers. (True)
3. HTML5 requires plugins like Flash to play video. (False)
4. The _____ meta tag is essential for responsive design on mobile devices. (viewport)
5. _____ attribute limits the maximum number of characters allowed in a text input. (maxlength)